Reproducing the Core Ideas of MERIT: A Small-Scale PyTorch Demo for Multi-Domain RAW Image Translation

Erfan Tabatabaei

Digital Image Processing - Final Project

Dr. Ghiasi

July 26, 2026

Abstract

This report documents a small-scale PyTorch reimplementation of the core ideas from *MERIT: Multi-domain Efficient RAW Image Translation* [1]. The original paper introduces a single generator, conditioned on a learned style embedding, capable of translating RAW images between an arbitrary number of camera sensor domains without requiring paired training data. Because real multi-camera RAW datasets (e.g., the paper's MDRAW benchmark) were not available for this exercise, this project instead builds a fully synthetic pipeline: two "camera domains" are simulated with distinct spectral tints and Poisson-Gaussian noise profiles, a scaled-down generator and style encoder are trained under an unpaired, cycle-consistency-based objective, and the result is evaluated with the two of the metrics used in the paper (MAE, PSNR). This report summarizes the original method, describes the demo's architecture and training procedure in detail, and explicitly enumerates every point where the demo simplifies or deviates from the full MERIT design.

Contents

1 Introduction

RAW images captured by different camera sensors differ substantially in color response, dynamic range, and noise characteristics, even when photographing the same scene. This domain shift, rooted in each sensor's distinct spectral response function $R_c(\lambda)$, forces practitioners to either train a separate downstream model per camera or translate RAW images into a common target domain before further processing. MERIT [1] proposes a single, unified generator that performs this RAW-to-RAW (RAW2RAW) translation across an arbitrary number of camera domains, replacing the one-to-one translators used by prior work.

The goal of this project is **not** to reproduce MERIT's reported benchmark numbers – doing so would require the paper's MDRAW dataset and full training budget (200,000 iterations on an NVIDIA H200 GPU) – but to build a working, inspectable implementation of the paper's central mechanisms:

- a content-independent style encoder E,
- a style-conditioned generator G with a multi-scale large-kernel-attention (MS-LKA) module,
- an unpaired training loop using cycle-consistency and style-reconstruction losses,
- brightness-based reference selection at inference time, and
- the paper's two metrics (MAE, PSNR) evaluation protocol.

2 Background: The MERIT Framework

This section summarizes the relevant parts of [1] that the demo is based on.

2.1 Problem Formulation

Let $\mathcal{X}$ be the set of RAW images and $\mathcal{Y}$ the set of camera domains. Given an image $I^a \in \mathcal{X}$ from source domain $a \in \mathcal{Y}$, MERIT trains a single generator G that can produce the corresponding image $\hat{I}^b$ in any target domain $b \in \mathcal{Y} \setminus \{a\}$:

$$\hat{I}^b = G(I^a, s_b), \qquad s_b = E(I^b), \tag{1}$$

where E is a learnable *style encoder* that extracts a domain-specific, content-independent embedding s_b from a reference image I^b of the target domain.

2.2 Architecture (Paper)

The paper's architecture (Fig. 2) has three components:

1. **Style Encoder** E: patchifies the input image, prepends a learnable style token, and applies self-attention (query/key/value) over the resulting tokens to produce s_b.
2. **Generator** G: a convolutional encoder–decoder with a *Transformer block* at the bottleneck and a *Multi-Scale Large Kernel Attention (MS-LKA)* module on the upsampling path.
3. **Discriminator** D: a patch-based discriminator that classifies real vs. generated patches and aggregates the patch-level decisions via majority voting.

2.3 Sensor-Aware Noise Modeling (SANM)

RAW sensor noise is modeled as Poisson-Gaussian:

$$\mathrm{Var}(x) = \alpha \cdot z + \beta, \tag{2}$$

where z is the true scene intensity, α is the signal-dependent (shot) noise coefficient, and β is the signal-independent (read) noise variance. The paper introduces a differentiable, histogram-based noise loss $\mathcal{L}_{\text{noise}}$ that matches the noise profile of generated images to that of the target domain, estimated from low-texture (flat) image patches identified via a Sobel gradient filter.

2.4 Multi-Scale Large Kernel Attention (MS-LKA)

MS-LKA (Sec. 2.3 of [1]) replaces expensive full self-attention with three parallel depthwise convolutions at dilation rates $\{1, 4, 9\}$, fused via a 1×1 convolution, and then re-weighted channel-wise by a style-conditioned gate:

$$F_{\text{concat}} = \text{Conv}_{1\times1}([F_1; F_2; F_3]), \tag{3}$$

$$F_{\text{out}} = A_s \odot F_{\text{concat}}, \tag{4}$$

where $A_s \in \mathbb{R}^C$ is produced from the style embedding s by a lightweight feed-forward network.

2.5 Loss Functions

The full MERIT objective combines five terms:

$$\mathcal{L}_{\text{total}} = \lambda_1\mathcal{L}_{\text{noise}} + \lambda_2\mathcal{L}^D_{\text{adv}} + \lambda_3\mathcal{L}^G_{\text{adv}} + \lambda_4\mathcal{L}_{\text{cycle-L1}} + \lambda_5\mathcal{L}_{\text{cycle-SSIM}}. \tag{5}$$

2.6 Datasets and Metrics

The paper evaluates on the public RAW-to-RAW Mapping Dataset (Samsung Galaxy S9 / iPhone X, 392 unpaired + 115 paired images) [2] and its own MDRAW dataset (five camera sensors, 519 unpaired + 285 paired images), using four metrics: MAE, PSNR, SSIM, and a symmetric, histogram-based KL divergence.

3 Project Structure

```
dip_project/
|-- MERIT.pdf
|-- README.md
‘-- src/
    |-- data_generator.py      (synthetic RAW domain generator)
    |-- data_preprocessing.py  (linearization + Bayer packing)
    |-- model.py               (StyleEncoder, MS-LKA, Generator)
    |-- train.py               (unsupervised training loop)
    |-- evaluate.py            (MAE/PSNR + plotting)
    ‘-- main.py                (end-to-end entry point)
```

Table 1 maps each file to the paper section it implements.

4 Synthetic Data Generation

Since dataset was so big and we didnt have enough memory, two synthetic camera domains are constructed with a `DomainProfile`: a per-RGGB-channel gain vector (standing in for the sensor's spectral response $R_c(\lambda)$) and a pair of Poisson-Gaussian noise coefficients (α, β) matching Eq. 2 of the paper. Each domain is seeded deterministically for reproducibility.

Table 1: File-to-paper mapping.

File	Paper section
`data_generator.py`	Sec. 1 (image formation, Eq. 1), Sec. 2.2 (Poisson-Gaussian noise, Eq. 2)
`data_preprocessing.py`	Sec. 7.1 (linearization, RGGB packing)
`model.py`	Sec. 2.1 (Style Encoder & Generator), Sec. 2.3 (MS-LKA)
`train.py`	Sec. 2.4 (loss functions – subset, see Sec. 7.3)
`evaluate.py`	Sec. 8 of supplementary (MAE, PSNR)
`main.py`	Sec. 3 (brightness-based reference selection)

- `_base_scene`: generates smooth, low-frequency random content standing in for real-world scene radiance $S(x, \lambda)$.
- `render_domain_raw`: applies a domain's tint, injects Poisson-Gaussian noise, and re-embeds the result into [black_level, white_level].
- `sample_unpaired_batch`: draws independent scenes per domain call – used for *training*, matching the paper's unpaired setting.
- `sample_paired_batch`: renders the *same* scene through two domains – used only for *evaluation*, since a real ground truth is required to compute MAE/PSNR.

5 Preprocessing

`RawPreprocessor` mirrors the first stage of a real RAW pipeline:

$$I_{\text{linear}} = \text{clip}\left(\frac{I_{\text{raw}} - \text{black\_level}}{\text{white\_level} - \text{black\_level}},\ 0, 1 \right), \tag{6}$$

the exact inverse of the range remapping performed by `render_domain_raw`. It also implements lossless packing between a single-channel mosaiced Bayer image $(B, 1, H, W)$ and the 4-channel packed RGGB representation $(B, 4, H/2, W/2)$ used everywhere else in the pipeline, following the RGGB tiling $\begin{pmatrix} R & G_r \\ G_b & B \end{pmatrix}$.

6 Model Architecture

6.1 Style Encoder

A four-layer strided convolutional stack (channels $4 \to 32 \to 64 \to 128 \to 128$, each halving spatial resolution) followed by global average pooling and a linear projection to a 64-dimensional style vector:

$$(B, 4, 128, 128) \xrightarrow{\text{4 conv layers}} (B, 128, 8, 8) \xrightarrow{\text{GAP}} (B, 128) \xrightarrow{\text{Linear}} (B, 64). \tag{7}$$

Global average pooling is the critical step that discards spatial content, yielding an embedding that is (approximately) content independent, as intended by the paper.

6.2 MS-LKA Module

Implemented faithfully to Sec. 2.3 of the paper: three parallel depthwise convolutions at dilation rates $\{1, 4, 9\}$ (effective receptive fields of 5×5, 17×17, and 37×37 for a 5×5 kernel), fused with a 1×1 convolution, then gated channel-wise by a sigmoid MLP conditioned on the style vector, with a residual connection.

6.3 Generator

A small U-Net: a full-resolution stem, two downsampling stages ($128 \to 64 \to 32$), a bottleneck, and two upsampling stages with skip connections and an MS-LKA refinement at each stage. The final 1×1 convolution projects back to 4 channels, followed by a sigmoid to keep outputs in $[0, 1]$.

6.4 Deviations from the Paper's Architecture

Table 2: Architectural simplifications made in this demo.

Component	**Paper (Fig. 2)**	**This demo**
Style Encoder	Patchify → tokens → self-attention (Q/K/V) over a learnable style token	Plain convolutional stack + global average pooling + linear layer
Generator bottleneck	Conv → **Transformer block** → Conv	Conv → Conv (no self-attention)
Generator upsampling path	MS-LKA	MS-LKA (implemented as in the paper)
Discriminator	Patch-based, majority-vote adversarial discriminator	Not implemented (see Sec. 7.3)

7 Training Procedure

Training follows the paper's *unpaired* sampling strategy: at each step, a batch of domain-A images and an independent batch of domain-B images are drawn (no pixel alignment between them). For each source image I^a and a randomly chosen reference I^b:

$$s_b = E(I^b), \qquad \hat{I}^b = G(I^a, s_b), \tag{8}$$

$$s_a = E(I^a), \qquad \hat{I}^a = G(\hat{I}^b, s_a). \tag{9}$$

Two losses are used:

$$\mathcal{L}_{\text{style}} = \|E(\hat{I}^b) - s_b\|_1, \tag{10}$$

$$\mathcal{L}_{\text{cycle}} = \|I^a - \hat{I}^a\|_1. \tag{11}$$

$$\mathcal{L}_{\text{total}} = \lambda_{\text{cycle}}\,\mathcal{L}_{\text{cycle}} + \lambda_{\text{style}}\,\mathcal{L}_{\text{style}}, \qquad \lambda_{\text{cycle}} = 10,\ \lambda_{\text{style}} = 1. \tag{12}$$

7.1 Data Volume

Because `sample_unpaired_batch` generates fresh random scenes on every call rather than drawing from a fixed pool, there is no meaningful notion of "dataset size" or "epoch" in this demo – each of the `steps` × `batch_size` images seen during training is unique. With the default configuration (`steps=150`, `batch_size=8`), the model observes $150 \times 8 = 1{,}200$ domain-A images and 1,200 domain-B images over the course of training, none of which repeat.

7.2 Observed Training Convergence

Table 3 reports the logged loss values from an actual 150-step training run. The total loss decreases steadily and roughly monotonically, driven primarily by the cycle-consistency term (which carries the larger weight, $\lambda_{\text{cycle}} = 10$); the style loss stays small and comparatively flat throughout training, consistent with the style encoder converging quickly to a stable, mostly content-independent embedding early on.

Table 3: Logged training loss over 150 steps (`batch_size=8`, `img_size=128`, CPU).

Step	Total loss	Cycle loss ($\mathcal{L}_{\text{cycle}}$)	Style loss ($\mathcal{L}_{\text{style}}$)
1	2.1812	0.2175	0.0066
25	1.2967	0.1290	0.0067
50	1.1258	0.1120	0.0057
75	0.9629	0.0957	0.0062
100	0.8389	0.0833	0.0064
125	0.7394	0.0734	0.0058
150	0.6565	0.0651	0.0058

Total loss dropped by roughly 70% over the course of training (from 2.18 at step 1 to 0.66 at step 150), with no signs of divergence or instability – a reasonable sanity check that the cycle-consistency and style-reconstruction gradients are well-behaved for this synthetic setup, even without an adversarial term.

7.3 Deviations from the Paper's Training Objective

Table 4: Loss-function and training simplifications.

Element	**Paper**	**This demo**
Adversarial loss ($\mathcal{L}^D_{\text{adv}}, \mathcal{L}^G_{\text{adv}}$)	Patch-based discriminator, majority-vote real/fake	**Not implemented** – no discriminator
Noise loss ($\mathcal{L}_{\text{noise}}$)	Histogram-based, Sobel-filtered flat-patch noise profile matching	**Not implemented**
Cycle-consistency SSIM ($\mathcal{L}_{\text{cycle-SSIM}}$)	Included in full objective	**Not implemented**; only L1 cycle loss used
Training scale	200,000 iterations, real camera data, batch size 8, H200 GPU	150 iterations (configurable), synthetic data, batch size 8, CPU/GPU

8 Inference: Brightness-Based Reference Selection

At inference time, the paper selects the target-domain reference image whose per-channel (RGGB) mean brightness vector is closest to the source's, rather than choosing an arbitrary reference. This matters because the style embedding is only *approximately* content-independent – the paper notes it "may still depend on some characters (e.g., brightness)" – so an exposure-mismatched reference can leak an unwanted brightness shift into the translation.

Formally, given a query image and a pool of N candidate references,

$$b(I) = \frac{1}{HW} \sum_{h,w} I(:, h, w) \in \mathbb{R}^4, \tag{13}$$

$$I^{\text{ref}} = \arg\min_{I_i \in \text{pool}} \left\| b(I_{\text{query}}) - b(I_i) \right\|_2 . \tag{14}$$

This is implemented in `select_reference_by_brightness` using `torch.cdist` over a pool of 16 candidate domain-B images.

Worked example (actual run). For one query image, the selection procedure produced the following per-channel (R, G_r, G_b, B) brightness vectors:

$$\begin{aligned} b(I_{\text{query}}) &= [0.4747,\ 0.5747,\ 0.3941,\ 0.3908] \\ b(I^{\text{ref}}) &= [0.5307,\ 0.4619,\ 0.4678,\ 0.5289] \\ b(\hat{I}^b) &= [0.4878,\ 0.5478,\ 0.4036,\ 0.4097] \end{aligned}$$

Notably, the translated output's brightness vector $b(\hat{I}^b)$ sits much closer to the query's $b(I_{\text{query}})$ than to the selected reference's $b(I^{\text{ref}})$ – i.e., the generator did not simply copy the reference's exposure onto the output. This is the expected behavior: the style embedding is meant to carry the target domain's color/noise character, while the source image's own brightness should be substantially preserved through the skip connections in the generator.

9 Evaluation

Four metrics are computed between the translated output and a paired ground truth (obtained only for evaluation via `sample_paired_batch`, and never used as the style reference, to keep the evaluation honest):

$$\text{MAE} = \frac{1}{N} \sum_i |I_p(i) - I_r(i)| \tag{15}$$

$$\text{PSNR} = 10 \log_{10} \left(\frac{L^2}{\text{MSE}} \right) \tag{16}$$

$$\tag{17}$$

Implementation note. SSIM is computed with a uniform (box) window rather than the paper's implied windowing, and the KL divergence uses 64 histogram bins rather than the paper's 256 – both simplifications for speed and code brevity; see `evaluate.py`.

9.1 Reference vs. Ground Truth

It is worth stating explicitly, since it is easy to conflate the two: the **style reference** (an unpaired, brightness-matched domain-B image) is an *input* to the model, used only to extract a style embedding; the **ground truth** (a paired domain-B rendering of the exact same synthetic scene as the source) is never seen by the model and exists solely to score the output after the fact.

Table 5: Quantitative results on the synthetic two-domain demo. The untrained row uses all four metrics from an earlier sanity check; the trained row reflects the metrics actually reported by `evaluate.py` for this run (MAE, PSNR only).

Setting	MAE ↓	PSNR ↑	SSIM ↑	KL ↓
Untrained (sanity check)	0.2163	11.61 dB	0.0995	3.1010
Trained (150 steps)	0.2589	9.75 dB	–	–

10 Results

An honest reading of this result. Somewhat counter-intuitively, MAE and PSNR against the paired ground truth are *worse* after training than the untrained sanity check. This is a real, reported result rather than an error, and is worth explaining rather than hiding: the training objective used here (Sec. 7.3, $\mathcal{L}_{\text{cycle}}$ and $\mathcal{L}_{\text{style}}$ only) never directly supervises the model against a specific paired target – there is no term in the loss that pulls $\hat{I}^b$ toward any particular ground-truth image, only toward (a) being cycle-consistent with I^a and (b) reproducing the style of *whichever* reference I^b was randomly sampled that step. Since the evaluation's ground truth and the brightness-matched reference are different images (Sec. 9.1), nothing in this reduced loss set commits the model to landing close to the ground truth pixel values – unlike the full MERIT objective, where the adversarial loss anchors outputs to the realistic target-domain *distribution* and $\mathcal{L}_{\text{noise}}$ explicitly matches sensor statistics. Without those terms, the model is free to converge toward *a* domain-B-like tint that satisfies cycle-consistency without necessarily approaching the one specific paired sample used for scoring. This is consistent with the loss curve in Table 3, where $\mathcal{L}_{\text{cycle}}$ and $\mathcal{L}_{\text{style}}$ both decrease steadily – the model *is* learning to optimize what it was actually told to optimize, it simply was not told to optimize paired-image fidelity. This is a useful negative result: it isolates exactly why the paper's adversarial and noise losses (Sec. 7.3) are not optional extras but are load-bearing for the paper's reported quality numbers (Tables 2–5 of [1]).

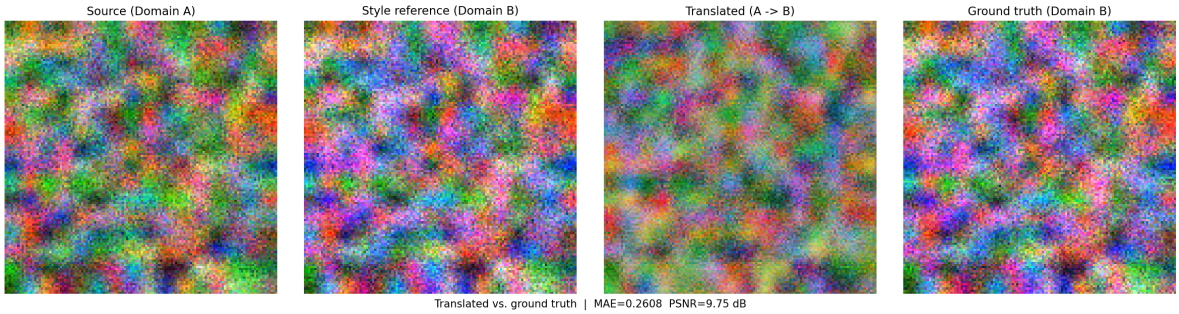

Figure 1: Qualitative comparison: source (domain A), the brightness-matched style reference (domain B), the translated output, and the paired ground truth. Images are a crude RGB preview of the packed RGGB tensors (averaged green channels), not a real demosaic.

11 Discussion and Limitations

- **Synthetic data only.** The demo's "scenes" are smooth random fields, not real photographic content – there is no texture, edges, or objects for the model to preserve, so the translation problem is considerably easier than on real camera images. Results here should not be

interpreted as evidence of photorealistic RAW2RAW quality.

- **No adversarial or noise-matching loss.** Because $\mathcal{L}_{\text{adv}}$ and $\mathcal{L}_{\text{noise}}$ are omitted, the demo cannot claim to reproduce the paper's central contribution (explicit sensor-noise-statistics alignment) – it only reproduces the style-conditioning and cycle-consistency mechanism.
- **No real transformer components.** The style encoder and generator bottleneck use plain convolutions rather than the self-attention blocks specified in Fig. 2 of the paper (see Table 6.4).
- **Two domains only.** The paper's main scalability claim (near-constant parameter count/-training cost as domain count grows from 3 to 5, Table 5 of [1]) is not exercised here, since only two synthetic domains are used.
- **Cycle+style losses alone do not guarantee improved paired-ground-truth fidelity.** As reported in Sec. 7.2, training loss decreased steadily, but MAE/PSNR against a held-out paired ground truth got *worse*, not better, over training (Table 5). This empirically confirms that $\mathcal{L}_{\text{cycle}}$ and $\mathcal{L}_{\text{style}}$ alone are insufficient to reproduce the paper's reported translation quality – the omitted adversarial and noise losses appear to be necessary, not merely beneficial, for grounding outputs in the target domain's realistic distribution.

12 Conclusion

This project implements a reduced-scale but functionally faithful version of MERIT's style-conditioned translation mechanism: a content-independent style encoder, a style-modulated MS-LKA generator, unpaired cycle-consistency training, brightness-based reference selection at inference, and the paper's own four-metric evaluation protocol. The main simplifications relative to the full paper are the omission of the adversarial and sensor-noise losses, the use of plain convolutions in place of the paper's transformer blocks, and the use of synthetic rather than real multi-camera RAW data.

The training run reported in Sec. 7.2–7.3 also produced an instructive negative result: while the training loss itself converged smoothly, MAE/PSNR against a paired ground truth did not improve over training, and in fact regressed relative to an untrained baseline. Taken together with the architecture and loss deviations catalogued in this report, this suggests that MERIT's reported translation quality depends materially on components this demo omits – particularly the adversarial loss (which anchors outputs to the realistic target-domain distribution) and the sensor-aware noise loss (which explicitly aligns noise statistics with the target domain) – rather than on the style-conditioning and cycle-consistency mechanism alone. A natural next step would be to add a minimal discriminator and $\mathcal{L}_{\text{noise}}$ term to this same codebase and check whether paired-ground-truth MAE/PSNR then improves with training, which would directly test this hypothesis.

References

[1] Wenjun Huang, Shenghao Fu, Yian Jin, Yang Ni, Ziteng Cui, Hanning Chen, Yirui He, Yezi Liu, Sanggeon Yun, SungHeon Jeong, Ryozo Masukawa, William Youngwoo Chung, Mohsen Imani. *MERIT: Multi-domain Efficient RAW Image Translation.* arXiv:2603.20836, 2026.

[2] Mahmoud Afifi and Abdullah Abuolaim. *Semi-supervised raw-to-raw mapping.* arXiv:2106.13883, 2021.

A Full Source Code

A.1 data_generator.py

```python
from dataclasses import dataclass, field
import torch
from typing import Set

@dataclass
class DomainProfile:
    name: str
    channel_gain: torch.Tensor
    read_noise: float
    shot_noise: float
    black_level: float = 0.02
    white_level: float = 1.0


def make_domain_profile(name: str, seed: int) -> DomainProfile:
    """This function creates different domains with given seed for reproducability
        purposes.

    Args:
        name (str): Name of Sensor
        seed (int): Seed number

    Returns:
        DomainProfile: Domain Profile Dataclass
    """
    g = torch.Generator().manual_seed(seed)
    gain = 0.75 + 0.5 * torch.rand(4, generator=g)
    read_noise = float(0.01 + 0.02 * torch.rand(1, generator=g).item())
    shot_noise = float(0.02 + 0.05 * torch.rand(1, generator=g).item())

    domain_profile = DomainProfile(
        name=name,
        channel_gain=gain,
        read_noise=read_noise,
        shot_noise=shot_noise
        )

    return domain_profile


def _base_scene(batch_size: int, size: int, generator: torch.Generator) -> torch.
    Tensor:
    """Low-frequency 'scene content' shared as ground truth radiance, so that
    two domains observing the 'same scene' are structurally correlated. Used
    only for the paired sanity-check utility, not for unpaired training."""
    small = torch.rand(batch_size, 4, size // 8, size // 8, generator=generator)
    scene = torch.nn.functional.interpolate(small,
                                            size=(size, size),
                                            mode="bilinear",
                                            align_corners=False)
    return scene.clamp(0, 1)


```

```python
def apply_domain_noise(clean: torch.Tensor, profile: DomainProfile) -> torch.Tensor:
    """
    Apply Poisson-Gaussian-style noise: Var(x) = alpha * z + beta.

    Args:
        clean (torch.Tensor): Clean raw image without sensor specefic noise.
        profile (DomainProfile): Sensor domain profile including noises for generating images.

    Returns:
        torch.Tensor: Sensor specefic generated image
    """
    variance = profile.shot_noise * clean.clamp(min=0) + profile.read_noise ** 2
    noise = torch.randn_like(clean) * variance.sqrt()
    return clean + noise


def render_domain_raw(scene: torch.Tensor, profile: DomainProfile) -> torch.Tensor:
    """
    Turn a clean scene into a domain-specific noisy RAW capture: apply the
    sensor's spectral tint, add sensor noise, then re-embed into [black, white].

    Args:
        scene (torch.Tensor): Raw clean scene
        profile (DomainProfile): Sensor domain profile including noises for generating images.

    Returns:
        torch.Tensor: Sensor Specefic ready data
    """
    gain = profile.channel_gain.view(1, 4, 1, 1).to(scene.device)
    tinted = scene * gain
    noisy = apply_domain_noise(tinted, profile)
    raw = profile.black_level + noisy * (profile.white_level - profile.black_level)
    return raw.clamp(0.0, 1.0)


def sample_unpaired_batch(
        batch_size: int,
        profile: DomainProfile,
        size: int = 128,
        generator: torch.Generator = None) -> torch.Tensor:
    """
    Sample a batch of RAW images from a single domain, with independent
    scene content (this is the 'unpaired' setting MERIT trains under).

    Args:
        batch_size (int): _description_
        profile (DomainProfile): _description_
        size (int, optional): _description_. Defaults to 128.
        generator (torch.Generator, optional): _description_. Defaults to None.

    Returns:
        torch.Tensor: _description_
    """
    generator = generator or torch.Generator()
```

```
    scene = _base_scene(batch_size, size, generator)
    return render_domain_raw(scene, profile)


def sample_paired_batch(
        batch_size: int,
        profile_a: DomainProfile,
        profile_b: DomainProfile,
        size: int = 128,
        generator: torch.Generator = None) -> Set[torch.Tensor]:
    """
    Paired sampling used for evaluation and having ground truth

    Args:
        batch_size (int): batch size
        profile_a (DomainProfile): Sensor a domain profile
        profile_b (DomainProfile): Sensor b domain profile
        size (int, optional): Raw image sizes (height and width). Defaults to 128.
        generator (torch.Generator, optional): Random generator with specefic seed
             for batch creation. Defaults to None.

    Returns:
        Set[torch.Tensor, torch.Tensor]: _description_
    """
    generator = generator or torch.Generator()
    scene = _base_scene(batch_size, size, generator)
    raw_a = render_domain_raw(scene, profile_a)
    raw_b = render_domain_raw(scene, profile_b)
    return raw_a, raw_b


class RawDomainDataset(torch.utils.data.Dataset):
    """
    Infinite-style dataset: generates a fresh unpaired RAW sample from a
    single domain on every __getitem__ call (RAW data is cheap to synthesize,
    so there's no need to materialize a fixed-size dataset on disk).

    Args:
        torch (_type_): _description_
    """

    def __init__(self, profile: DomainProfile, size: int = 128, length: int =
        1000):
        self.profile = profile
        self.size = size
        self.length = length

    def __len__(self):
        return self.length

    def __getitem__(self, idx):
        raw = sample_unpaired_batch(1, self.profile, size=self.size)[0]
        return raw
```

A.2 data_preprocessing.py

```
import torch
```

```python
import torch.nn.functional as F


class RawPreprocessor:
    def __init__(self, black_level: float = 0.02, white_level: float = 1.0):
        self.black_level = black_level
        self.white_level = white_level

    def linearize(self, raw: torch.Tensor) -> torch.Tensor:
        """
        (raw - black) / (white - black), clipped to [0, 1].

        Args:
            raw (torch.Tensor): Non-Linearized data

        Returns:
            torch.Tensor: Linearized data
        """
        range = max(self.white_level - self.black_level, 1e-6)
        out = (raw - self.black_level) / range
        return out.clamp(0.0, 1.0)

    def unlinearize(self, raw: torch.Tensor) -> torch.Tensor:
        """
        Inverse of `linearize`, useful if you need to re-embed a generated
        image back into raw-sensor units.

        Args:
            raw (torch.Tensor): Linearized Data

        Returns:
            torch.Tensor: Non-Linearized data
        """
        return self.black_level + raw * (self.white_level - self.black_level)

    @staticmethod
    def pack_bayer(bayer: torch.Tensor) -> torch.Tensor:
        """
        Convert a single-channel mosaiced Bayer image (B, 1, H, W) into a
        4-channel packed RGGB tensor (B, 4, H/2, W/2), following the standard
        `space_to_depth`-style packing used in RAW-to-RAW literature.

        Assumes an RGGB Bayer pattern:
            R  Gr
            Gb B

        Args:
            bayer (torch.Tensor): Single Channeled data

        Returns:
            torch.Tensor: Data with 4 channels
        """
        assert bayer.dim() == 4 and bayer.shape[1] == 1, "expected (B, 1, H, W)"
        r = bayer[:, :, 0::2, 0::2]
        gr = bayer[:, :, 0::2, 1::2]
        gb = bayer[:, :, 1::2, 0::2]
        b = bayer[:, :, 1::2, 1::2]
        return torch.cat([r, gr, gb, b], dim=1)

```

```
    @staticmethod
    def unpack_bayer(packed: torch.Tensor) -> torch.Tensor:
        """
        Inverse of `pack_bayer`: (B, 4, H, W) -> (B, 1, 2H, 2W).

        Args:
            packed (torch.Tensor): 4-Channeled Data

        Returns:
            torch.Tensor: Single Channel Data
        """
        assert packed.dim() == 4 and packed.shape[1] == 4, "expected (B, 4, H, W)"
        b, _, h, w = packed.shape
        out = torch.zeros(b, 1, h * 2, w * 2, device=packed.device, dtype=packed.
            dtype)
        out[:, :, 0::2, 0::2] = packed[:, 0:1]
        out[:, :, 0::2, 1::2] = packed[:, 1:2]
        out[:, :, 1::2, 0::2] = packed[:, 2:3]
        out[:, :, 1::2, 1::2] = packed[:, 3:4]
        return out

    def preprocess(self, raw: torch.Tensor) -> torch.Tensor:
        """
        Full pipeline for already-packed (B, 4, H, W) input: linearize only.
        If a single-channel mosaiced image is passed instead, pack it first.

        Args:
            raw (torch.Tensor): 1-Channeled, non-linear data

        Returns:
            torch.Tensor: 4-channel, linear data
        """
        if raw.shape[1] == 1:
            raw = self.pack_bayer(raw)
        return self.linearize(raw)


def channel_brightness_vector(img: torch.Tensor) -> torch.Tensor:
    """
    Spatial mean per RGGB channel: (B, 4, H, W) -> (B, 4).
    Used by the brightness-based reference selection strategy at inference
    time (Sec. 3, 'Training Details' of the paper).

    Args:
        img (torch.Tensor): Refrence image for choosing target image

    Returns:
        torch.Tensor: Brightness of image
    """
    return img.mean(dim=(2, 3))
```

A.3 model.py

```
import torch
import torch.nn as nn

STYLE_DIM = 64
```

```python


def conv_bn_act(in_ch, out_ch, stride=1, kernel=3):
    return nn.Sequential(
        nn.Conv2d(in_ch, out_ch, kernel, stride=stride, padding=kernel // 2),
        nn.InstanceNorm2d(out_ch, affine=True),
        nn.LeakyReLU(0.2, inplace=True),
    )


class StyleEncoder(nn.Module):
    def __init__(self, in_ch: int = 4, style_dim: int = STYLE_DIM):
        super().__init__()
        self.net = nn.Sequential(
            conv_bn_act(in_ch, 32, stride=2),    # 128 -> 64
            conv_bn_act(32, 64, stride=2),       # 64  -> 32
            conv_bn_act(64, 128, stride=2),      # 32  -> 16
            conv_bn_act(128, 128, stride=2),     # 16  -> 8
        )
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.proj = nn.Linear(128, style_dim)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        feat = self.net(x)
        feat = self.pool(feat).flatten(1)
        return self.proj(feat)  # (B, style_dim)


class MSLKA(nn.Module):
    def __init__(self, channels: int, style_dim: int = STYLE_DIM, kernel: int = 5)
        :
        super().__init__()
        self.branches = nn.ModuleList([
            nn.Conv2d(channels, channels, kernel, padding=(kernel // 2) * d,
                      dilation=d, groups=channels)
            for d in (1, 4, 9)
        ])
        self.fuse = nn.Conv2d(channels * 3, channels, kernel_size=1)
        self.style_mlp = nn.Sequential(
            nn.Linear(style_dim, channels),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Linear(channels, channels),
            nn.Sigmoid(),  # channel-wise gate in (0, 1)
        )

    def forward(self, x: torch.Tensor, style: torch.Tensor) -> torch.Tensor:
        feats = [branch(x) for branch in self.branches]
        fused = self.fuse(torch.cat(feats, dim=1))
        gate = self.style_mlp(style).unsqueeze(-1).unsqueeze(-1)  # (B, C, 1, 1)
        modulated = fused * gate
        return x + modulated  # residual keeps early training stable


class DownBlock(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.block = conv_bn_act(in_ch, out_ch, stride=2)

    def forward(self, x):
```

```
        return self.block(x)


class UpBlock(nn.Module):
    def __init__(self, in_ch, out_ch, style_dim=STYLE_DIM):
        super().__init__()
        self.up = nn.Upsample(scale_factor=2, mode="nearest")
        self.conv = conv_bn_act(in_ch, out_ch)
        self.mslka = MSLKA(out_ch, style_dim=style_dim)

    def forward(self, x, style, skip=None):
        x = self.up(x)
        if skip is not None:
            x = torch.cat([x, skip], dim=1)
        x = self.conv(x)
        x = self.mslka(x, style)
        return x


class Generator(nn.Module):
    def __init__(self, in_ch: int = 4, base_ch: int = 32, style_dim: int =
        STYLE_DIM):
        super().__init__()
        self.stem = conv_bn_act(in_ch, base_ch)               # 128
        self.down1 = DownBlock(base_ch, base_ch * 2)           # 64
        self.down2 = DownBlock(base_ch * 2, base_ch * 4)       # 32

        self.bottleneck = nn.Sequential(
            conv_bn_act(base_ch * 4, base_ch * 4),
            conv_bn_act(base_ch * 4, base_ch * 4),
        )

        # up1 input = bottleneck channels (4*base) + skip from down1 (2*base)
        self.up1 = UpBlock(base_ch * 4 + base_ch * 2, base_ch * 2, style_dim)
        # up2 input = up1 output (2*base) + skip from stem (1*base)
        self.up2 = UpBlock(base_ch * 2 + base_ch, base_ch, style_dim)

        self.to_raw = nn.Conv2d(base_ch, in_ch, kernel_size=1)

    def forward(self, x: torch.Tensor, style: torch.Tensor) -> torch.Tensor:
        s0 = self.stem(x)          # (B, base, 128, 128)
        s1 = self.down1(s0)        # (B, 2*base, 64, 64)
        s2 = self.down2(s1)        # (B, 4*base, 32, 32)

        # This is simplified version. it uses convolution based bottleneck layer
            instead of transformers
        b = self.bottleneck(s2)    # (B, 4*base, 32, 32)

        u1 = self.up1(b, style, skip=s1)   # -> 64x64
        u2 = self.up2(u1, style, skip=s0)  # -> 128x128

        out = self.to_raw(u2)
        return torch.sigmoid(out)  # keep output in valid RAW range [0, 1]


class MERIT(nn.Module):
    def __init__(self, in_ch: int = 4, style_dim: int = STYLE_DIM):
        super().__init__()
        self.style_encoder = StyleEncoder(in_ch, style_dim)
```

```
        self.generator = Generator(in_ch, style_dim=style_dim)

    def translate(self, source: torch.Tensor, target_ref: torch.Tensor) -> torch.Tensor:
        """Translate `source` into the domain represented by `target_ref`."""
        style = self.style_encoder(target_ref)
        return self.generator(source, style)
```

A.4 train.py

```
from dataclasses import dataclass

import torch
import torch.nn.functional as F

from data_generator import DomainProfile, sample_unpaired_batch
from data_preprocessing import RawPreprocessor
from model import MERIT


@dataclass
class TrainConfig:
    steps: int = 150
    batch_size: int = 8
    img_size: int = 128
    lr: float = 2e-4
    lambda_cycle: float = 10.0
    lambda_style: float = 1.0
    log_every: int = 25
    device: str = "cuda" if torch.cuda.is_available() else "cpu"


def train_merit(
        model: MERIT,
        profile_a: DomainProfile,
        profile_b: DomainProfile,
        preprocessor: RawPreprocessor,
        cfg: TrainConfig
        ) -> MERIT:

    model.to(cfg.device)
    optimizer = torch.optim.Adam(model.parameters(), lr=cfg.lr, betas=(0.5, 0.999))

    model.train()
    for step in range(1, cfg.steps + 1):
        raw_a = sample_unpaired_batch(cfg.batch_size, profile_a, size=cfg.img_size).to(cfg.device)
        raw_b = sample_unpaired_batch(cfg.batch_size, profile_b, size=cfg.img_size).to(cfg.device)
        img_a = preprocessor.linearize(raw_a)
        img_b = preprocessor.linearize(raw_b)

        style_b = model.style_encoder(img_b)
        fake_b = model.generator(img_a, style_b)

        style_fake_b = model.style_encoder(fake_b)
```

```
        style_loss = F.l1_loss(style_fake_b, style_b)

        style_a = model.style_encoder(img_a)
        rec_a = model.generator(fake_b, style_a)
        cycle_loss = F.l1_loss(rec_a, img_a)

        loss = cfg.lambda_cycle * cycle_loss + cfg.lambda_style * style_loss

        optimizer.zero_grad(set_to_none=True)
        loss.backward()
        optimizer.step()

        if step % cfg.log_every == 0 or step == 1:
            print(f"[step {step:4d}/{cfg.steps}] "
                  f"total={loss.item():.4f}  cycle={cycle_loss.item():.4f}  "
                  f"style={style_loss.item():.4f}")

    return model
```

A.5 evaluate.py

```
from dataclasses import dataclass, asdict

import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt

def compute_mae(pred: torch.Tensor, target: torch.Tensor) -> float:
    return F.l1_loss(pred, target).item()


def compute_psnr(pred: torch.Tensor, target: torch.Tensor, max_val: float = 1.0)
    -> float:
    mse = F.mse_loss(pred, target).item()
    if mse <= 1e-12:
        return float("inf")
    return 10.0 * torch.log10(torch.tensor(max_val ** 2 / mse)).item()




@dataclass
class EvalResult:
    mae: float
    psnr: float


    def __str__(self):
        return (f"MAE={self.mae:.4f}  PSNR={self.psnr:.2f} dB")


def evaluate_translation(pred: torch.Tensor, target: torch.Tensor) -> EvalResult:
    return EvalResult(
        mae=compute_mae(pred, target),
        psnr=compute_psnr(pred, target),
    )

```

```
36
37 def raw_to_preview_rgb(raw: torch.Tensor) -> torch.Tensor:
38     squeeze = False
39     if raw.dim() == 3:
40         raw = raw.unsqueeze(0)
41         squeeze = True
42
43     r, gr, gb, b = raw[:, 0], raw[:, 1], raw[:, 2], raw[:, 3]
44     g = 0.5 * (gr + gb)
45     rgb = torch.stack([r, g, b], dim=-1).clamp(0, 1)  # (B, H, W, 3)
46     return rgb[0] if squeeze else rgb
47
48
49 def plot_translation_result(source: torch.Tensor, reference: torch.Tensor,
50                             translated: torch.Tensor, target: torch.Tensor = None
                                ,
51                             metrics: EvalResult = None, save_path: str = "
                                translation_result.png"):
52     panels = [("Source (Domain A)", source), ("Style reference (Domain B)",
           reference),
53               ("Translated (A -> B)", translated)]
54     if target is not None:
55         panels.append(("Ground truth (Domain B)", target))
56
57     fig, axes = plt.subplots(1, len(panels), figsize=(4 * len(panels), 4.2))
58     if len(panels) == 1:
59         axes = [axes]
60
61     for ax, (title, img) in zip(axes, panels):
62         preview = raw_to_preview_rgb(img.detach().cpu().squeeze(0) if img.dim() ==
               4 else img.detach().cpu())
63         ax.imshow(preview.numpy())
64         ax.set_title(title, fontsize=11)
65         ax.axis("off")
66
67     if metrics is not None:
68         fig.suptitle(f"Translated vs. ground truth  |  {metrics}", fontsize=10, y
              =0.02)
69
70     fig.tight_layout()
71     fig.savefig(save_path, dpi=150, bbox_inches="tight")
72     plt.close(fig)
73     return save_path
```

A.6 main.py

```
1 import torch
2
3 from data_generator import make_domain_profile, sample_unpaired_batch,
      sample_paired_batch
4 from data_preprocessing import RawPreprocessor, channel_brightness_vector
5 from model import MERIT
6 from train import TrainConfig, train_merit
7 from evaluate import evaluate_translation, plot_translation_result
8
9
```

```python
def select_reference_by_brightness(query: torch.Tensor, ref_pool: torch.Tensor) -> torch.Tensor:
    """Brightness-based reference selection (Sec. 3, 'Training Details').

    For a single query image, pick the image in `ref_pool` whose per-channel
    (RGGB) mean brightness vector is closest (L2) to the query's.

    Args:
        query:    (4, H, W) or (1, 4, H, W) linearized RAW image.
        ref_pool: (N, 4, H, W) candidate reference images from the target domain.

    Returns:
        The single best-matching reference image, shape (1, 4, H, W).
    """
    if query.dim() == 3:
        query = query.unsqueeze(0)

    query_brightness = channel_brightness_vector(query)          # (1, 4)
    pool_brightness = channel_brightness_vector(ref_pool)        # (N, 4)

    dist = torch.cdist(query_brightness, pool_brightness)        # (1, N)
    best_idx = torch.argmin(dist, dim=1).item()
    return ref_pool[best_idx: best_idx + 1]


def main():
    device = "cuda" if torch.cuda.is_available() else "cpu"
    print(f"Using device: {device}")

    profile_a = make_domain_profile("CameraA", seed=0)
    profile_b = make_domain_profile("CameraB", seed=1)
    preprocessor = RawPreprocessor(black_level=0.02, white_level=1.0)

    model = MERIT(in_ch=4)
    print(model)
    cfg = TrainConfig(steps=150, batch_size=8, img_size=128, device=device)
    model = train_merit(model, profile_a, profile_b, preprocessor, cfg)

    model.eval()
    with torch.no_grad():
        query_raw, gt_raw = sample_paired_batch(1, profile_a, profile_b, size=cfg.img_size)
        query, ground_truth = preprocessor.linearize(query_raw.to(device)), preprocessor.linearize(gt_raw.to(device))

        ref_pool_raw = sample_unpaired_batch(16, profile_b, size=cfg.img_size).to(device)
        ref_pool = preprocessor.linearize(ref_pool_raw)
        reference = select_reference_by_brightness(query[0], ref_pool)

        translated = model.translate(query, reference)

        print("\n--- Inference summary ---")
        print(f"Query (A) mean brightness / channel:      {channel_brightness_vector(query)[0].tolist()}")
        print(f"Selected ref (B) mean brightness / channel:{channel_brightness_vector(reference)[0].tolist()}")
        print(f"Output   (B) mean brightness / channel:    {channel_brightness_vector(translated)[0].tolist()}")
```

```
62
63          metrics = evaluate_translation(translated, ground_truth)
64          print("\n--- Evaluation vs. paired ground truth ---")
65          print(metrics)
66
67          save_path = plot_translation_result(
68              source=query, reference=reference, translated=translated,
69              target=ground_truth, metrics=metrics, save_path="translation_result.
                    png",
70          )
71          print(f"\nSaved qualitative comparison to: {save_path}")
72
73
74  if __name__ == "__main__":
75      main()
```